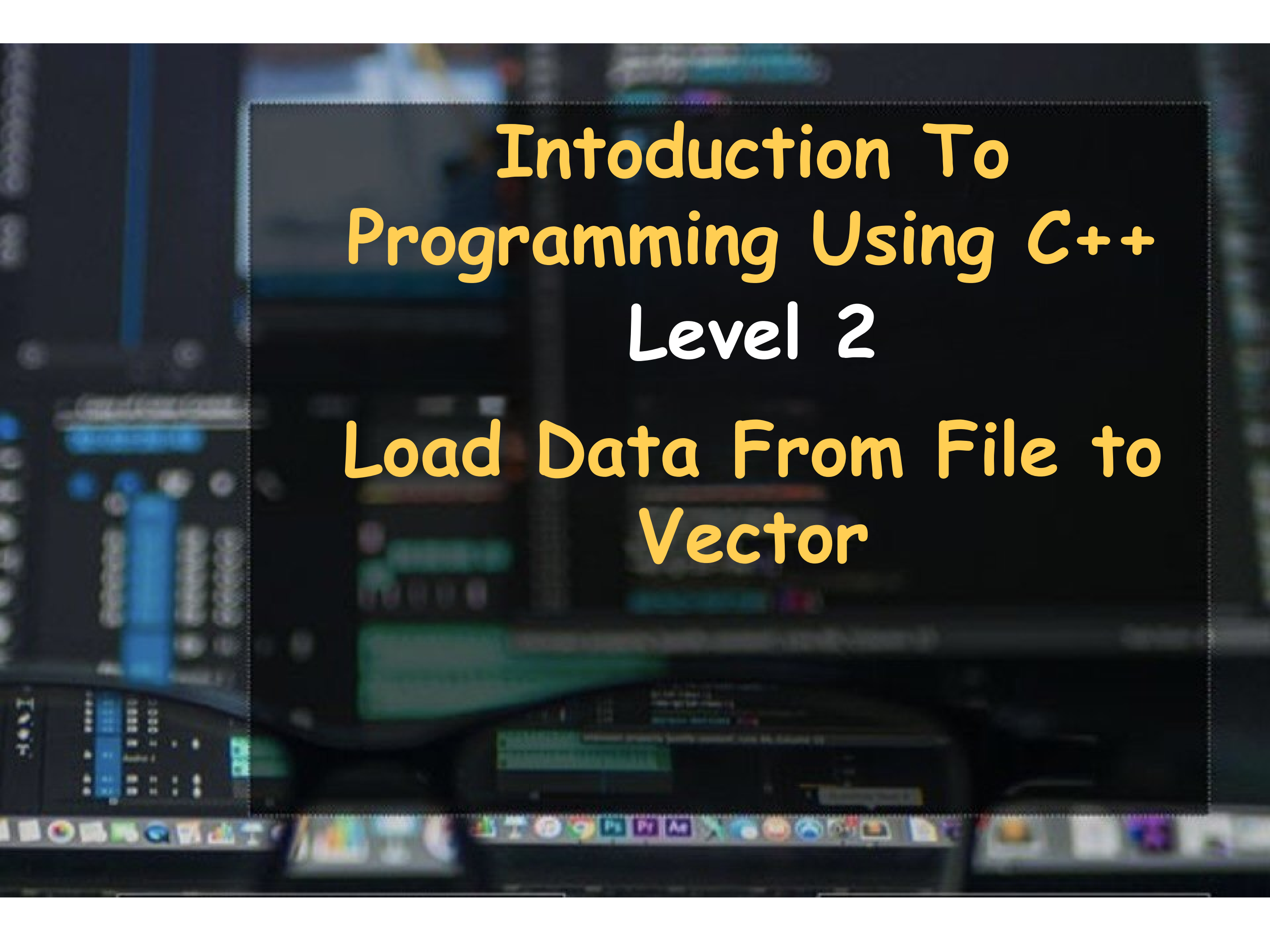


The background of the slide is a blurred image of a computer monitor. The monitor shows a dark-themed code editor with some text visible, and a taskbar at the bottom with various application icons. The text is overlaid on this background.

Introduction To Programming Using C++ Level 2

Load Data From File to Vector



Introduction To Programming Using C++

Level 2

Load Data From File to Vector

Definition

Load Data From File to Vector

 Loading data from a file into a `vector<string>` means reading the text file line by line and storing each line as an element inside the vector.

 This allows us to work with the data inside the program, such as modifying, deleting, adding, or processing elements, and later saving the data back to a file.

Explanation

Required Libraries

-  ``<iostream>`` → input and output.
-  ``<fstream>`` → working with files.
-  ``<string>`` → using ``string``.
-  ``<vector>`` → storing the file data.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
```

Function Definition

-  The function receives the file name as a ``string``.
-  It also receives the vector by reference ``&`` so the function can directly modify the original vector.

```
void LoadDataFromFileToVector(string fileName, vector<string>& vFileContent)
```

Opening the File

-  We open the file using ``ios::in`` because we want to read its contents.

```
fstream MyFile;
MyFile.open(fileName, ios::in);
```

Reading the File

-  We use ``getline()`` to read the file one line at a time.
-  Each line is stored in ``Line``.
-  ``push_back()`` adds the line to the end of the vector

Procedures

1. Include the required libraries.
2. Create a `vector<string>` to store the file contents.
3. Pass the vector by reference to the function.
4. Open the file using `ios::in`.
5. Check whether the file opened successfully.
6. Read each line using `getline()`.
7. Add each line to the vector using `push_back()`.
8. Close the file after finishing.

💡 Example

```
void LoadDataFromFileToVector(string fileName, vector<string>& vFileContent)
{
    fstream MyFile;
    MyFile.open(fileName, ios::in);    // Read Mode
    if (MyFile.is_open())
    {
        string Line;
        while (getline(MyFile, Line))
        {
            vFileContent.push_back(Line);
        }
        MyFile.close();
    }
}

int main()
{
    vector<string> vFileContent;
    LoadDataFromFileToVector("MyFile.txt", vFileContent);
    for (string vElement : vFileContent)
    {
        cout << vElement << endl;
    }
    return 0;
}
```

💡 Example

➡ Result

If `MyFile.txt` contains:

Ali

Shadi

Maher

Fadi

Lama

The vector will contain:

{"Ali", "Shadi", "Maher", "Fadi", "Lama"}

And the program prints:

Ali

Shadi

Maher

Fadi

Lama

Note

- ◆ The vector is passed by reference using `&` so the function modifies the original vector from `main()`.
- ◆ `getline()` reads one complete line at a time.
- ◆ `push\_back()` adds each read line to the end of the vector.
- ◆ This approach separates file storage from the data we work with inside the program.

⊘ Warning

- ◆ If the file does not exist, `ios::in` does not create it.
- ◆ Always check `is_open()` before reading.
- ◆ Passing the vector by value would create a copy, so changes made inside the function would not affect the original vector.
- ◆ If the same vector is loaded multiple times without clearing it, the new data will be added to its existing elements.

✦ Summary

- ✓ Open the file using `ios::in`.
- ✓ Read each line using `getline()`.
- ✓ Store each line using `push_back()`.
- ✓ Pass the vector by reference when the function needs to modify it.
- ✓ Close the file after reading.
- ✓ The result is a `vector<string>` containing the file data.
- ✓ The vector can then be modified or processed inside the program before being saved again.

Thank you 😊